

LLM-Assisted Development Workflow (Jan 2026)

Introduction: AI coding assistants have become game-changers in software development, but harnessing them effectively requires skill, structure, and a clear process ¹. As an experienced developer, I have fully embraced an *AI-augmented software engineering* approach ² – treating the Large Language Model (LLM) as a powerful pair programmer that needs clear direction, context, and human oversight ³. This document outlines my current workflow for developing with LLMs (notably using Anthropic's Claude as my assistant, though the approach works with any advanced LLM ⁴). The workflow is *not* about letting the AI take over blindly; rather, it's a disciplined method where the developer's expertise guides the AI at every step. It assumes you can read and design code yourself – the LLM amplifies an expert, but isn't a substitute for fundamental development skills ⁵. By combining classic software engineering practices (planning, architecture design, testing) with LLM capabilities, this workflow yields a **dramatic productivity boost** while maintaining code quality and developer control.

Workflow Overview: At a high level, my development process with an LLM consists of the following stages:

1. **Define Requirements & Gather Context:** Start with a clear problem or feature definition and assemble only the most relevant code context for the LLM – don't dump the whole codebase.
2. **Iterative Planning with the LLM:** Engage in a deep conversation with the LLM to brainstorm architecture and refine the solution. Flesh out requirements, design decisions, and edge cases through iterative prompts.
3. **Develop a Detailed Specification (Brief):** Collaboratively compile a comprehensive *spec* or brief that captures exactly what to build – including design choices, data models, interfaces, and examples – as the blueprint for coding.
4. **Code Generation from the Spec:** Provide the spec (and any needed guidance docs) to the LLM and have it implement the code. The AI writes actual code files for the feature based on the agreed specification.
5. **Testing, Debugging, and Refinement:** Next, have the LLM produce unit tests for the new code. Integrate the code into the codebase and run tests one by one, fixing any issues discovered. Iterate with the LLM or manual edits to refine the code until all tests pass and the solution is clean.

Throughout these steps, **human oversight is paramount** – I review everything, run and read tests, and ensure I fully understand the output before considering the task done ⁶ ⁷. Below, I'll dive into each stage in detail.

1. Defining Requirements and Providing Focused Context

The process begins with a well-defined requirement or feature idea. Rather than asking the LLM to “build X” with no context, I **clearly describe the problem and goal** upfront. This might include a short spec of the feature's purpose, any constraints, and how it should fit into the existing system. Starting with a clear problem definition ensures both the human and AI are “*on the same page*” before any coding begins ⁸.

Equally important at this stage is **providing the right context without overloading the model**. LLMs are highly sensitive to the input they receive, and giving them *too much irrelevant information can hurt*

performance ⁹ ¹⁰. Instead of dumping an entire repository or large documents into the prompt, I **strategically gather only the pieces of code and documentation that are relevant to the task at hand** ¹¹ ¹². For example, if I'm adding a new API endpoint, I might pull in the existing interface definition, related data model classes, and perhaps a similar endpoint's implementation as reference. I avoid flooding the LLM with files or details unrelated to the current feature.

Why not give the whole codebase? Studies and industry experience show that providing an LLM with *too much context* can confuse it and even degrade output quality. The phenomenon known as "*Lost in the Middle*" illustrates how models can actually miss important details when they're buried in the middle of an overly long prompt ¹³. In practice, overwhelming the AI with information leads to token waste, slower responses, and sometimes a "*confused output*" where the model struggles to find the signal in the noise ¹². As one AI engineer put it, dumping a huge codebase into the prompt can produce results like "*10 devs worked on it without talking to each other*" – inconsistent and duplicative code requiring major refactoring ¹⁴.

To avoid this, I use a **context engineering** approach: curate the prompt content *intelligently*. I give the model "*only what matters — the right information, in the right format, at the right time.*" ¹¹ ¹⁵ This often means **delivering context in phases** as we progress (sometimes called *context phasing* ¹⁶): I'll start the conversation with just the core pieces needed to brainstorm the solution, and introduce additional code snippets or docs later when they become relevant. This focused context keeps the LLM's attention where it needs to be and fits within token limits.

Practically speaking, I have a custom tool that helps assemble context from the codebase. Given a repository or folder path, it can generate a single text file containing the contents of multiple relevant files. This way, I can quickly provide, say, 10–20 source files worth of content in one prompt if needed, without manually copying each file. It's similar to using open-source utilities like *repo2txt* or *gitingest*, which "*dump the relevant parts of your codebase into a text file for the LLM to read*", making large projects easier to handle ¹⁷. The key is that *I* (the developer) decide which parts of the codebase to include, based on my understanding of the requirement. By anchoring the LLM's focus on a carefully chosen subset of code and context, I ensure it starts from the right foundation. Modern LLMs have large context windows (tens or even hundreds of thousands of tokens), but wise use of that space is crucial – include enough to give the AI what it needs, but **not so much that the core facts get lost** ¹⁰ ⁹.

It's worth noting that this workflow assumes the developer is familiar with the codebase and architecture. I'm essentially playing the role of a curator for the AI, selecting the useful references for the task. This stands in contrast to handing the LLM an entire repo and hoping it figures out relevance. "*Context engineering isn't about stuffing more into the prompt — it's about curating smarter.*" ¹¹ By giving the model high-signal, low-noise input, we set it up for success in the next stages.

2. Iterative Planning and Architecture Brainstorming

Once the initial problem and minimal context are provided, I move into an **interactive planning dialogue** with the LLM. Rather than jumping straight to code, the LLM and I brainstorm the solution approach in depth *before* any implementation. This is analogous to a developer talking through a design with a colleague – except the colleague is the AI, which can offer suggestions, catch edge cases, and even enumerate alternative approaches.

I usually prompt the LLM to **help flesh out the requirements and design**. For example, I might say: "*Here's what I want to build... Given the context above, let's discuss how to implement this. What modules or*

classes would we need? Any potential pitfalls? Please ask any questions to clarify the requirements." This kicks off a back-and-forth conversation. Often the LLM will ask insightful questions (e.g. about how a new feature should handle certain inputs or integrate with existing components) or it will start proposing an outline of a solution. We effectively perform a technical **Q&A and design session**. This step is crucial because coding is more than just writing lines – it's about understanding the problem and mapping out a solution. With an AI assistant, I can quickly probe the "art of the possible" and leverage its knowledge of software design patterns or libraries that might fit.

During this phase, we **iterate heavily** on the plan. I might realize from the LLM's questions that I forgot to specify a requirement, or the LLM might initially suggest an approach that isn't ideal, prompting me to steer it in a different direction. I keep refining the prompt with additional details or corrections ("Actually, we should store data X in this format, not that," or "Assume we have a service Y available for this part"). The LLM, in turn, refines its understanding and updates the design proposal. We continue this loop – adjusting and elaborating – until a clear architecture and solution approach emerges that I'm satisfied with.

Crucially, **I treat the LLM's suggestions as just that – suggestions**. The AI can be very confident even when it's wrong, so I always vet its ideas against my own knowledge and best practices. In other words, the LLM is like a very fast-thinking junior architect: it can generate ideas and consider many factors quickly, but I validate each decision. This aligns with the industry guideline that an LLM pair programmer is *"over-confident and prone to mistakes"*, requiring the human to stay critical and in charge ¹⁸. I never blindly accept an architectural choice without scrutiny; if something sounds off, we discuss further or I override it.

By the end of this intensive brainstorming, what we have is essentially a detailed game plan for the implementation. In traditional terms, I've collaboratively produced a **design document** or **technical specification**. In fact, this practice is echoed by other LLM-powered developers: they spend significant time upfront on planning with the AI, because *"planning first forces you and the AI onto the same page and prevents wasted cycles."* ⁸ It's tempting to skip straight to coding, but experienced LLM users treat a robust spec/plan as **the cornerstone of the workflow** ⁸. As one developer quipped, this step is like doing a *"waterfall in 15 minutes"* – a rapid structured design phase that makes the later coding much smoother ¹⁹.

3. Developing a Detailed Specification (The "Brief")

After the interactive planning conversation, I ask the LLM to **summarize the agreed-upon solution into a detailed written specification**. This spec (or "brief") is a comprehensive description of *what* we are going to implement and *how*. It typically runs several hundred lines long (in complex cases, even 1000+ lines) and reads like a well-documented design plan. The brief usually includes:

- **High-level feature description:** What we're building and why, in plain language.
- **Architecture and components:** A breakdown of the modules, classes, functions, or services involved, and how they interact.
- **Data models and schemas:** Definitions of any new data structures, database schemas, or API payloads, etc.
- **Function signatures and interfaces:** Outlines of key functions or methods (sometimes written as pseudo-code or code stubs), showing inputs/outputs.
- **Expected behaviors and edge cases:** Descriptions of how the system should behave under various scenarios, including error handling and edge conditions.

- **Integration points:** Notes on how new components integrate with existing systems (for example, “this function calls X API,” or “extends Y base class”).
- **Non-functional requirements or constraints:** e.g. performance considerations, security implications, or compliance with certain patterns.
- **Testing strategy:** Plans for how we will verify the feature (unit tests to write, example scenarios to test, etc.).

This document is effectively the **contract** for the implementation. I often find that by the time we finish writing the spec, large parts of the code solution are already sketched out in it. In fact, the spec is so detailed that one *could* almost start coding from it directly (and in some cases, I might manually implement straightforward parts from the spec while reading it). Nonetheless, having the AI write out the spec is valuable for a few reasons:

- **Clarity and correctness:** It forces the LLM to articulate the plan clearly. Any ambiguities or misunderstandings from the chat get flushed out when we see them written down. I review the spec carefully and often spot places to tweak. If something in the spec doesn’t match what I intended, I correct it now (either by editing or by telling the LLM to adjust that part). The spec serves as a final sanity check that the AI and I are 100% aligned before coding.
- **Single-source of truth for prompts:** Once the spec is polished and accurate, it becomes the primary input for the coding phase. I can use it to prompt the LLM to generate code, ensuring consistency. Since the spec encapsulates all our decisions, the AI can refer back to it instead of juggling the entire prior conversation history. This is especially useful if I start a **fresh LLM session for coding** – I can just paste the spec in, and the new session is immediately grounded in the plan without needing the whole dialogue.

It’s important to mention that at this stage I might bring in **additional reference materials** to include in the spec if not already covered. For instance, if I have internal coding standards or a specific library usage guide, I might add a section in the spec summarizing those rules. I’ve built up a set of “guidance docs” (for coding style, type safety patterns, testing conventions, etc.) and I incorporate the relevant parts into the spec so that the code generation will adhere to them. This is akin to what many developers do by providing style guides or rule files to the AI – *“steer your AI assistant by providing style guides, examples... upfront tuning yields much better outputs.”*²⁰. By customizing the AI’s context with these rules and examples, I greatly improve the chances that the generated code will match our standards and frameworks out of the gate.

In summary, by the end of this phase I have a **thorough blueprint** of the solution. Both I and the LLM “know exactly what we’re building and why,” and we’ve ironed out potential issues in advance⁸. This upfront investment in a detailed spec may feel time-consuming, but it *“pays off enormously”* when it comes to coding¹⁹ – just as writing a good design doc makes traditional development smoother. Now, with the spec (brief) in hand, we’re ready to turn it into actual code.

4. Code Generation and Implementation with the LLM

With a comprehensive spec prepared, the next step is **using the LLM to generate the code implementation**. I will either continue in the same chat session or (often) start a fresh session with the model specifically for code generation. Beginning fresh can be advantageous to ensure the model’s focus is on the final spec alone, and it helps avoid any confusing context from the earlier Q&A. In the new session, I essentially give the model one big prompt: I provide the detailed spec (and any supplemental guidance docs if needed) and then instruct, *“Now implement this.”*

Thanks to all the groundwork laid, the LLM is now able to produce code that closely follows the spec. In many cases, **the AI will generate multiple files or modules in one go**, since the spec already delineated what files/functions are needed. For example, if the plan called for a new class, a corresponding data model, and an update to an existing module, the LLM might output all of those pieces together. Modern coding models (especially ones with large context windows like Claude 2 or GPT-4) can handle surprisingly extensive outputs. I've had the LLM generate on the order of 10, 20, or even 40 files worth of code in a single response when the task was comprehensive. (When outputs are this large, Claude might stream its answer for several minutes – but it does manage to complete it.)

I usually **let the LLM attempt the entire feature's implementation in this pass**. This contrasts with an alternative approach where you might break the coding into smaller sub-tasks (implement one function at a time, etc.). Both strategies can work; breaking into chunks is safer if the model has limited context or if the problem is very open-ended ²¹ ¹⁴ . However, since I already have a concrete spec, I find it efficient to let the AI draft the whole thing at once, guided by that spec. Essentially, the spec acts like a very detailed set of instructions that keeps the model on track, even though the output is large. This way I get a complete initial implementation to evaluate.

Tip – Choosing the right model: In my experience, it helps to use the most capable model available for the actual coding step, because code generation benefits from both understanding and sheer output length. I personally lean on Claude for many coding tasks, as I've found it often *“understands better what I'm trying to do”* and can produce lengthy, coherent code outputs without losing the thread. Others have reported success with models like GPT-4 for similar reasons. The general principle is to **pick your tool with intention** – not all coding LLMs are equal, so choose one that fits the task, and don't hesitate to swap models if needed mid-stream ⁴ . For instance, one might use one model to draft code and another to review it (I sometimes have Claude write the code and then ask a GPT-based model to critique it, leveraging their different strengths ²²). The workflow is model-agnostic: whether the LLM runs on a cloud service or locally on your machine, as long as it understands code and has sufficient context length, it can be integrated into this process.

When the LLM provides the code output, I carefully **inspect what it has given**. Generally, because the model was following a spec I co-wrote, the structure of the code meets my expectations – classes and functions are present as planned, and the logic is roughly in line with the design. There are often small mistakes or omissions (no AI is perfect), but at this point I don't expect flawless code. The goal was to get *“90% of what I need”* quickly in code form, which I can then refine.

Depending on the situation, I might do a quick **read-through of the AI's code** immediately to catch any obvious issues. If I notice something clearly wrong (e.g. it misunderstood a requirement or there's a logical error), I will address it before moving on. Minor fixes I can do myself (just like editing code written by a human colleague), or I might ask the LLM in the chat to fix a specific part (“Please correct function X to do Y as per the spec”). At this stage, the LLM serves as a fast code author, but *I remain the code's custodian*, ensuring it aligns with our design and quality standards.

Once I'm comfortable that the AI's output is a reasonable first draft of the feature, I proceed to integrate the new code into my development environment for the next phase: testing and verification.

5. Testing, Debugging, and Refinement

Code complete, right? Not quite – now comes a critical part of the workflow: **testing and debugging the AI-generated code**. Just as in traditional development, new code isn't "done" until it's validated. In fact, with AI-written code, testing is even more crucial because the AI might introduce subtle errors or oversights a human might not. My workflow builds testing in as a first-class citizen, and I even use the LLM to help with it.

Generating tests with the LLM: After getting the implementation code, I typically prompt the LLM to **write unit tests** for the new functionality. (If the spec already included a detailed testing plan or examples, the AI often anticipates this, but I explicitly ask for tests to ensure thorough coverage.) I provide any testing-specific guidelines at this point – for example, I might remind the AI of the testing framework to use or the style of tests we prefer. Because the AI has context of the code it just wrote (and the spec), it can produce tests that cover the expected behaviors. I usually request one test file per major module or class created. The result is a suite of tests that essentially document how the code should work.

This practice of having the LLM draft tests has multiple benefits. First, it's fast – we can get dozens of test cases in minutes, something that could take a human much longer. At Assembled, an engineering team reported that using LLMs to generate comprehensive test suites **"in minutes"** was a game changer, significantly reducing the effort to create tests and ensuring they weren't skipped for lack of time ²³. Second, the tests often act as a second pair of eyes on the implementation: if the AI wrote a test for a scenario it forgot to handle in code, that signals a bug or missing logic. In essence, the AI is checking its own work by writing tests.

Of course, **human oversight is still required**. I never just trust the AI's tests blindly – I read through the test code to verify it makes sense. LLMs are pretty good at generating plausible tests, but they might occasionally assert the wrong expected value or miss an edge case. One team noted that you may need a few iterations to cover all edge cases and that engineers must *"read and run any LLM-generated tests before merging"*, just to double-check logic ²⁴. I follow the same rule: I inspect the tests and adjust any assertions if they don't align with the spec. If an important case is missing, I can ask the AI to add a test for it or I write it myself.

Running tests and debugging: With the new code and its test suite in my local environment, I run the tests one by one. This is where the real verification happens. Typically, some tests will fail initially – which is expected and *welcome*, because it means the tests are doing their job uncovering issues. Each failing test gives insight into what's wrong: perhaps a function returns `None` instead of an empty list, or an error isn't handled as expected. I use these failures to **iteratively debug and refine the code**.

My process for debugging is straightforward: I take the failing test and see why it failed (examining error messages or assertions), then trace that back to the code. Since I have just generated the code, it's fresh and relatively small in scope (thanks to our earlier focus). I often fix the issue myself on the spot – these are usually small logic tweaks or off-by-one type errors. If the fix is non-trivial or I want to ensure consistency, I might go back to the LLM: I'll describe the failing scenario and ask for a corrected snippet or an explanation. The LLM can be helpful in diagnosing issues too, almost like a rubber-duck debugger. For instance, if I'm puzzled by why a particular logic path is incorrect, I can show the AI the code and the test and ask it to analyze the discrepancy.

One by one, I go through the tests: run, fail, fix, and repeat. This sequence serves a dual purpose – it verifies the code and also **forces me to review the code thoroughly**. By debugging each test, I

naturally end up reading all parts of the AI-generated code, which ensures I truly understand and trust the final result. It's a deliberate *"human in the loop"* moment. As AI expert Simon Willison has said, an LLM coding partner is like an *"over-confident junior developer"* – you must rigorously review and test everything it produces ¹⁸. I take that to heart: *"always review and test thoroughly"* ⁶, treating the AI's output as if I'd gotten it from a human co-worker. If something doesn't make sense, I question it and refine it.

This test-driven refinement loop continues until **all tests pass** and I'm satisfied with the code quality. By the end, the AI-written code is typically in excellent shape: it meets the spec, and it's validated by a robust set of tests. The combination of AI generation plus stringent testing yields a very high confidence in the new feature. In fact, developers who employ strong testing practices find that *"an agent like Claude can 'fly' through a project with a good test suite as a safety net"* ²⁵ – the tests guide the AI and catch it when it falters. Conversely, without tests, the AI might blithely assume everything is fine when it's not ²⁵. That's why I invest in testing; it amplifies the AI's usefulness and ensures we catch mistakes.

Finally, I integrate the polished code into the main codebase (if I was working in a branch or separate environment). I make sure to commit the changes with clear messages. (Pro tip: frequent commits are useful when working with AI contributions, as they act as "save points" to revert to if needed ²⁶ ²⁷. I tend to commit after each significant fix or batch of changes, maintaining a clean history of what the AI produced and what I adjusted.)

At this point, the feature is complete: we have implementation and tests, all verified. But the power of this workflow doesn't stop at backend logic – I also apply it to other aspects of development, including front-end and integration tasks, which I'll discuss next.

6. Extended Applications: UI Prototyping and Multi-Step Workflows

The LLM-assisted workflow I've described isn't limited to core logic or APIs. I've found it equally useful for building **front-end components and even entire prototype user interfaces** to aid in testing and development. The idea is the same: give the AI a clear spec and let it generate code – whether that code is Python backend logic or HTML/JavaScript for a web UI.

For example, on a recent project I needed to test a complex data flow: a backend service that communicates with an external API via a middleman proxy. Simulating this purely with unit tests was challenging because it involved interactive steps and asynchronous behavior. My solution was to create a **simple UI tool** – essentially a small web frontend – to manually run and visualize the interactions in a browser. Using the LLM, I followed the iterative approach to develop this UI: I wrote a brief describing how the frontend should call my backend, what data to display, and how to allow me to trigger certain events step by step. This spec included example code for using the relevant web APIs and any specific frameworks (in this case, just a minimal HTML/JS setup with fetch calls to my local service).

I then initiated a new session with a model specialized in front-end generation (in principle I could use the same model, but sometimes I find switching helps if one model is better at a certain domain). Feeding in the UI spec, I asked the LLM to generate the necessary files – HTML, CSS, JavaScript – to implement the tool. Within a single response, the AI produced the markup and script for a functional interface according to my description. I integrated those files, opened them in a browser, and immediately had a working minimal UI to exercise the backend feature. A few tweaks were needed

(some styling, minor event handling adjustments), which I again used the AI to assist with in follow-up prompts.

This approach of quickly spinning up a UI or “flow tester” with the LLM is incredibly powerful. It turned what could have been a time-consuming manual coding task into a brief iteration. It’s a bit like having the AI as a *front-end developer* on demand. Not only does it speed up creating the interface, it also ensures the UI is aligned with the backend because the same spec was used. I’ve dubbed this approach **IFD – Iterative Flow Development** – where you iteratively build out a user flow with the AI’s help, much like we do for backend logic. In practice, it means the AI can help with end-to-end slices of functionality: backend, tests, and even the UI to invoke or demonstrate the feature.

Moreover, the workflow supports multi-step or multi-agent interactions. As noted earlier, one can use different LLM instances for different roles – e.g. one for writing code, another for reviewing or testing it ²². In my UI example, I had one session that was focused on backend logic and another that was focused on front-end. They were connected by the spec I wrote as a hand-off. This pattern can be generalized: you might have an LLM generate a design doc, then pass that to another LLM to generate code, then perhaps another to generate documentation or user manuals, etc. We’re essentially orchestrating a pipeline of AI assistants, each specializing in a part of the development lifecycle. It’s no surprise that leading developers now leverage AI at “*every point of the software lifecycle*”, from design and coding to testing and DevOps ²⁸. When done with careful coordination, this multi-agent workflow can dramatically accelerate development while still keeping the human in control of the overall process.

Conclusion and Key Takeaways

As of January 2026, this LLM-assisted development workflow has become my default way of building software. It allows me to create features rapidly and with a high degree of confidence in their quality. The key takeaways and principles from this approach are:

- **Start with a solid plan:** Don’t rush into coding. Use the AI to help **brainstorm requirements and design** first. A comprehensive spec or plan is the foundation for success ²⁹ ³⁰. It aligns the AI with your vision and prevents wasted effort.
- **Curate the context for the AI: Feed the model relevant information, not the kitchen sink.** Strategic context selection (a.k.a. context engineering) leads to better results than overwhelming the AI with too much data ¹¹ ⁹. Introduce knowledge in stages and focus the AI on the task at hand.
- **Use the LLM as a partner, not an oracle:** Treat the AI as a capable but fallible assistant – like a junior developer who can work very fast. **Maintain oversight:** review every output, question it, and guide the AI when it’s off-track ⁶ ¹⁸. Your judgment is crucial; the AI augments your productivity but doesn’t replace careful thinking ³.
- **Leverage AI in all phases (with appropriate tools):** From drafting specs to writing code to generating tests and even UI prototypes, *let the AI help*. Modern AIs can produce an astonishing range of outputs when given clear instructions. Choose the right model for each job (code generation, review, etc.) and don’t be afraid to use multiple AIs collaboratively ⁴ ²². Each can add value in its area of strength.
- **Integrate testing and validation at every step:** This workflow heavily emphasizes testing – often created by the AI itself – to catch issues early and give both you and the AI feedback ²⁵. Tests are not an afterthought but a core part of development here. Running those tests and debugging the results ensures you truly understand and trust the code. As a result, you get a *clean, safe codebase and faster development velocity* ²³, without sacrificing quality.

- **Continue to apply software engineering discipline:** Perhaps counterintuitively, the advent of AI coding tools makes good software practices even more important. Things like thoughtful design, modularization, code style consistency, and version control are what enable the AI to be effective. My experience echoes what others have noted: *“all our hard-earned practices – design before coding, write tests, use version control, maintain standards – not only still apply, but are even more important when an AI is writing half your code.”* ² In short, **AI doesn’t replace engineering discipline; it amplifies it.**

By following these principles, I’ve found that *the developer+AI duo is far more powerful than either alone* ⁵ ³¹. I can delegate the rote work of writing boilerplate or exhaustive tests to the LLM, freeing myself to focus on creative problem-solving and fine-tuning. Meanwhile, I stay in control of architectural decisions and critical thinking – the AI augments my capabilities but I remain *“the director of the show.”* ³²

This workflow has proven “ridiculously productive” in my day-to-day work. Features that might have taken days can often be spec’d, coded, and tested in mere hours with the AI’s help. Yet, I do so with confidence that the result is solid, because of the careful planning and verification baked into the process. It truly feels like having a supercharged pair programmer available 24/7. And as the tools improve (larger contexts, more integrated development agents, etc.), I anticipate this style of development will become even more powerful.

In conclusion, developing with LLMs in early 2026 means **working with the AI, not just asking it for code**. It’s a collaboration: I provide direction, expertise, and judgment; the LLM provides speed, knowledge, and draft output. Embracing this approach has allowed me to build software faster and better than ever, while still *“staying proudly accountable for the software produced”* ³³. As we move forward, I’m excited to keep refining this AI-assisted workflow – always learning, adapting, and ensuring that the human developer remains at the heart of the creative process.

Sources:

1. Addy Osmani – “My LLM coding workflow going into 2026”, *Medium*, Dec. 19, 2025 ¹ ³.
2. Addy Osmani – “My LLM coding workflow going into 2026” (sections on planning, chunking, context, etc.), *Medium*, Dec. 2025 ²⁹ ¹⁴ ¹⁷.
3. Rohit Biswal – “Context engineering: How to give AI exactly what it needs”, *Thoughtworks Blog*, Aug. 28, 2025 ¹¹ ¹⁶.
4. Suryadeepsinh Gohil – “The Context Window Paradox: Why Bigger Isn’t Always Better in AI”, *DEV Community*, Jan. 2024 ¹⁰ ¹³.
5. Addy Osmani – “Keep a human in the loop — verify, test, and review everything”, *Medium*, Dec. 2025 ⁶ ¹⁸.
6. Addy Osmani – “Keep a human in the loop — verify, test, and review everything”, *Medium*, Dec. 2025 ²⁵ ²².
7. Assembled Engineering – “How we saved hundreds of engineering hours by writing tests with LLMs”, Oct. 2023 ²⁴ ²³.
8. Addy Osmani – “Conclusion: AI-augmented software engineering”, *Medium*, Dec. 2025 ² ³².
9. Addy Osmani – “AI tools amplify your expertise”, *Medium*, Dec. 2025 ⁵.
10. Addy Osmani – “Provide extensive context and guidance”, *Medium*, Dec. 2025 ³⁴ ³⁵.

1 2 3 4 5 6 7 8 14 17 18 19 20 21 22 25 26 27 28 29 30 31 32 33 34 35 My LLM

coding workflow going into 2026 | by Addy Osmani | Dec, 2025 | Medium

<https://medium.com/@addyosmani/my-llm-coding-workflow-going-into-2026-52fe1681325e>

9 11 12 15 16 Context engineering: How to give AI exactly what it needs | Thoughtworks United States

<https://www.thoughtworks.com/en-us/insights/blog/generative-ai/context-engineering-give-ai-what-needs>

10 13 The Context Window Paradox: Why Bigger Isn't Always Better in AI - DEV Community

https://dev.to/suryadeepsinh_gohil/the-context-window-paradox-why-bigger-isnt-always-better-in-ai-1clj

23 24 How we saved hundreds of engineering hours by writing tests with LLMs

<https://www.assembled.com/blog/how-we-saved-hundreds-of-engineering-hours-by-writing-tests-with-llms>